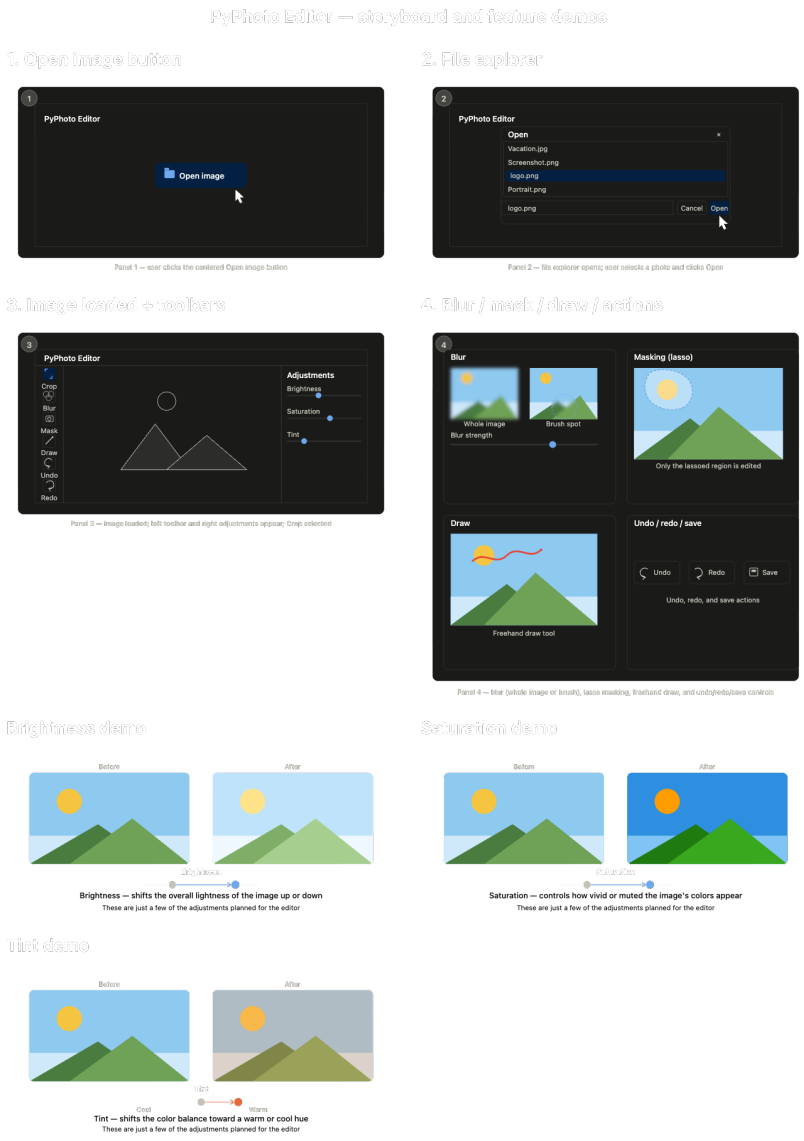
PyPhoto Editor

Description

PyPhoto Editor is a Python-based image editing application that lets users load, edit, and save their own images. It supports basic adjustments such as brightness, saturation, hue, and tint, along with other features such as blur, crop, basic drawing tools, and region-based masking. This masking system allows edits and filters to be applied selectively to specific parts of an image rather than the whole canvas.

Storyboard. Images created by AI.



Structural Plan

Notes written by hand and consolidated/summarized with AI. All thoughts are my own.

Overview: Three Layers

The project is organized into three layers, each with a distinct responsibility. This separation is intentional: it keeps image-processing logic (which needs to be algorithmically rich and independently testable) separate from state management (undo/redo, current document state) and from the UI (event handling, drawing to the screen). No layer needs to know the internal details of another layer's implementation in order to do its job.

- **Image Data & Processing Layer**: represents pixel data and performs the actual computations (blur, brightness, crop, etc.) that transform one image into another.
- **App / Document Layer**: manages the current state of the document being edited: the current image, the history of edits, and the logic to undo/redo/save.
- **UI Layer**: handles everything the user directly sees and interacts with: buttons, sliders, mouse events, and the visual feedback shown while a tool is in use.

App / Document Layer

This layer manages the current state of the editing session; what image is loaded, what has been done to it, and how the user's actions (open, undo, redo, save) translate into changes in that state.

Opening an image

- The user needs to be able to open an image file.
    - The file is selected via the file explorer (the operating system's native file-picker dialog).
    - Once selected, the image needs to be converted into a form that cmu_graphics can display
    - The image is then displayed on the canvas.

Saving progress and undo

- The user needs to be able to save their work, and to undo actions via an undo button.
    - To support undo, the system needs some mechanism to store both the previous state and the current state, so that a prior state can be restored on request.
    - The undo feature stores the **previous 10 states**.

        - Importantly, this does **not** mean storing 10 full copies of the entire image (which would be memory-inefficient). Instead, the system stores the **actions** that were performed (e.g., "brightness +20") and computes how to **reverse** each action (e.g., applying "brightness −20") rather than keeping a full pixel-for-pixel snapshot at each step. This might vary depending on the edit applied, as “undoing” a blur isn’t as straightforward as undoing brightness.

Exporting

- Exporting the image applies all of the edits that have been made to the image.
    - All edits made during the editing session are already applied and visible to the user as they work (what you see during editing is what you get).
    - Exporting takes the original image and applies the accumulated edits, producing the final output file.

UI Layer

This layer is responsible for what the user sees and how the user's clicks and drags are translated into actions elsewhere in the program.

Starting state

- The application has a defined starting state before any image is loaded.
    - The screen displays the title **"PyPhoto Editor."**
    - An **"Open image"** button is shown in the center of the screen.
        - When the mouse hovers over the button, a gray box appears around it to indicate it is interactive (hover feedback).

Once an image is loaded

- Once an image has been loaded, the full editing interface becomes available:
    - **Buttons** that give access to tool-based features: blur, masking, lasso, etc.
    - **Sliders** that give access to value-based adjustments: brightness, saturation, etc.
    - **Buttons** that give access to undo, redo, and save functionality.

How editing tools interact with the app

Each editing tool interacts with the app slightly differently in terms of what visual feedback it shows to the user, but there is an important shared pattern across all of them:

- **Blur** shows a transparent circle over the image, following the cursor, when the brush option is selected.
- **Draw** shows a circle/dot over the image following the cursor.
    - The size of that dot depends on the brush thickness the user has selected.
- **Lasso** works the same way as draw, but instead of a dot, it draws the growing polygon/outline as the user traces it over the image.

All three of these behaviors (blur-brush preview, draw preview, lasso preview) can be handled by one shared function, called with different parameters depending on which tool is currently selected rather than writing separate, duplicated logic for each tool.

Event timing: preview vs. commit

All of the above tool interactions are governed by a consistent event-based pattern, split across three mouse events:

- **onMousePress**: fires once, the moment the mouse button is first pressed down. This marks the start of a gesture (e.g., the starting point of a lasso).
- **onMouseDrag**: fires repeatedly while the mouse is moving with the button held down. This event is responsible **only** for drawing the live preview (the circle, dot, or growing polygon) — it does **not** modify the actual pixel data. Recomputing a real edit on every drag event would be computationally wasteful and would make the app feel sluggish.
- **onMouseRelease**: fires once, when the mouse button is released. This is the point at which the previewed edit is actually **committed**: the real pixel-changing computation runs exactly once, and the result (or the corresponding action, for undo purposes) is finalized.

This preview-then-commit pattern applies generally across every drag-based tool (blur-brush, draw, lasso), not just one specific case.

Image Data & Processing Layer

This layer is responsible for representing image data and for performing the actual computations that change the image. The deeper algorithmic details of how each of these operations works (e.g., the convolution math behind blur, the specific approach to flood-fill or lasso-based masking) are addressed separately in the Algorithmic Plan section, since that level of detail belongs there.

Planned processing components

- **Blur**: takes in pixel data and returns a blurred version of it.
- **Draw**: handles freehand drawing onto the image.
- **Crop**: handles cropping the image down to a selected region.

- **Overall editing/adjustments**: holds the current values for brightness, saturation, tint, and other slider-based adjustments, and applies them to the image data.
- **Possible future features** under consideration: subject selection and edge detection, filters, etc. time permitting.

Core data representation

- A dedicated class/structure is needed to store the actual pixel data itself, independent of any editing logic. This includes:
    - The RGB values for every pixel in the image.
    - The image's dimensions (width and height).

This class's only responsibility is to hold pixel data and allow it to be read and written, it does not need to know anything about what edits have been made, what tool is active, or how undo works. Any processing function (blur, brightness, etc.) should be able to operate on this class using only the pixel data itself.

Edit history

- A separate class is needed to represent the history of edits made to the image.
    - This class holds the edit history … the ordered record of actions taken (e.g., "brightness +20", "blur applied to region X") that is used to support undo/redo.
    - This is kept entirely separate from the pixel-data class described above: the pixel grid does not need to know about past actions in order to answer a question like "what color is this pixel right now," and the history does not need to know about raw pixel values in order to track what actions occurred.

Coordinator

- A third component is responsible for connecting the pixel-data class and the edit-history class, since neither of them should reach into the other directly.
    - It asks the history for the most recent action when an undo is requested.
    - It figures out the reverse of that action (e.g., turning "brightness +20" into "brightness −20").
    - It tells the pixel-data class to update itself using that reversed action, applying the change to the actual RGB values.

This coordinator is the only piece that needs to know about both the current pixel data and the edit history… keeping the responsibilities of the other two classes clean and independent, and making each of them easier to reason about, test, and extend as new features (additional filters, edge detection, subject selection, etc.) are added later.

Algorithmic Plan

Gaussian Blur:

1. Build the raw kernel
    a. For a chosen size, depending on the user input for gaussian blur strength, we increase the sigma value in the gaussian blur formula and increase the size of the kernel to get a more averaged look
    b. Loop through every row and column position in the kernel we construct (3x3) (5x5) etc, represented as a 2d List
2. Normalize the kernel so all the weights add up to one
    a. Sum all raw values, divide each individual value by that sum.
3. For each output pixel, using its position plus each kernel cell offset to find the corresponding neighboring input pixel
    a. Multiply each neighboring pixel's value by its corresponding kernel weight
    b. Sum all those weighted products together. This is the new value for that output pixel
    c. Do this separately for R, G, B
4. Handle edges
    a. Rough idea is to mirror the interior pixels and run the gaussian kernel
5. Loop through every pixel in the image, or for the brush case, every pixel in the brush area to produce a blurred output

Lasso tool:

1. Store drag points as a list
2. On release, close the loop by adding one final segment that connects the last recorded point to the first recorded point
3. That closed list of points is the polygon
4. For every pixel, shoot a ray in any direction, calculate how many times it intersects the edge of the polygon
5. If numIntersection % 2 == 0, then its outside, if numIntersection % 2 == 1, then it is inside the polygon
    a. This is a raycasting algorithm
6. Loop through every pixel, for the ones that have an odd number of intersections, add them to a list
7. (Known limitation, closing line that happens to cross the polygon).

CMU Graphics
Pillow
Not sure if I will need Tkinter to open files

Timeline Plan

TP1 (due Monday 7/27) Working code covering basic features, plus progress on the most complex feature:

- Full image open/display pipeline working end-to-end (file picker → Pillow → PixelImage → CMUImage → shown on canvas).
- Brightness adjustment working (the simplest slider-based edit, proving out the adjustment → apply → redraw pipeline before tackling harder adjustments).
- Convolution engine and Gaussian blur built and tested standalone (called directly against test images to verify correctness), not yet required to be wired into the UI.
- Deliberately deferred: undo/redo, lasso, drawing, crop, saturation/tint. Undo/redo specifically is deferred until multiple real edit types exist, so that the action-reversal logic can be tested against varied, real cases rather than only the simplest one (brightness).

TP2 (due Wednesday 7/29) — Working demo, "basically works" but need not be complete or polished:

- Blur fully integrated into the UI: both whole-image and brush-based options, with live preview during drag and the actual blur committed once on release.
- Saturation and tint added alongside brightness, using the same adjustment pipeline.
- Freehand draw tool working, using the same press/drag/release event pattern as blur.
- Non-destructive crop working.
- Undo/redo built, now that brightness, saturation, tint, blur, draw, and crop all exist to test reversal logic against (including resolving whether/how a blur action can be cleanly reversed).
- Lasso: at minimum, the ray-casting mask logic built and tested standalone; full UI integration (live dashed-polygon preview, constraining edits to the masked region) as a stretch goal if time allows.

TP3 (due Friday 7/31) — Final polish:

- Lasso fully integrated into the UI, if not already completed at TP2.
- Save/export finalized.
- Bug fixing and UI polish.
- README, video demo, and shortcut commands prepared for grading and the final oral.